ARTIFICIAL INTELLIGENCE

LABORATORY RECORD

COURSE CODE: CSA1712

EXPERIMENTS 11–15

Name:S.Lekha
Reg no:192511104

EXPERIMENT 11: MAP COLORING USING CSP

Question: Write the Python program for Map Coloring to implement CSP.

Aim: To solve the map-coloring problem using a Constraint Satisfaction Problem (CSP) by assigning different colors to adjacent regions.

Algorithm

1. Represent each region as a node and its adjacent regions as neighbors.
2. Select a region and try assigning an available color.
3. Check that no adjacent region has the same color.
4. Use backtracking if a color assignment leads to a conflict.
5. Display the assignments when a valid solution is found.

Program

```python
def map_coloring(graph, colors):
    result = {}
    regions = list(graph.keys())

    def solve(index):
        if index == len(regions):
            return True

        region = regions[index]
        for color in colors:
            if all(result.get(n) != color for n in graph[region]):
                result[region] = color
                if solve(index + 1):
                    return True
                del result[region]
        return False

    if solve(0):
        print("Map Coloring:")
        for region, color in result.items():
            print(region, ":", color)
    else:
        print("No solution exists")

graph = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}
map_coloring(graph, ['Red', 'Green', 'Blue'])
```

Sample Output

```
Output

Map Coloring:
A : Red
B : Green
C : Blue
D : Red
```

Result

The map-coloring problem was solved using CSP and backtracking. No two adjacent regions are assigned the same color.

EXPERIMENT 12: TIC TAC TOE GAME

Question: Write the Python program for Tic Tac Toe game.

Aim: To develop a two-player Tic Tac Toe game in Python where players take turns placing X and O on a 3 × 3 board.

Algorithm

6. Initialize a board with nine empty positions.
7. Display the board and request a position from 1 to 9.
8. Validate the chosen position and place the player's symbol.
9. Check rows, columns, and diagonals for a winning combination.
10. Display the winner or declare a draw when the board is full.

Program

```
board = [' '] * 9

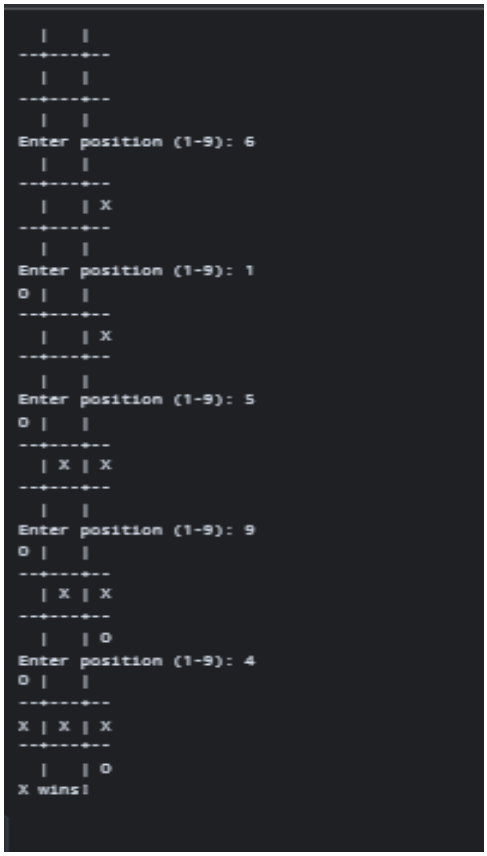
def display():
    for i in range(0, 9, 3):
        print(board[i], '|', board[i+1], '|', board[i+2])
        if i < 6:
            print('--+---+--')

def winner():
    lines = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)
    ]
    for a, b, c in lines:
        if board[a] == board[b] == board[c] != ' ':
            return board[a]
    return None

player = 'X'
turn = 0
while turn < 9:
    display()
    pos = int(input("Enter position (1-9): ")) - 1
    if pos < 0 or pos > 8 or board[pos] != ' ':
        print("Invalid position! Try again.")
        continue

    board[pos] = player
    turn += 1
    if winner():
        display()
        print(player, "wins!")
        break
    player = 'O' if player == 'X' else 'X'
else:
    display()
    print("Draw!")
```

Sample Output



Result

The two-player Tic Tac Toe game was implemented. It checks winning combinations and displays a winner or a draw.

EXPERIMENT 13: MINIMAX ALGORITHM FOR GAMING

Question: Write the Python program to implement Minimax algorithm for gaming.

Aim: To implement the Minimax algorithm for a two-player game tree and determine the best value for the maximizing player.

Algorithm

11. Represent possible game outcomes as leaf values in a game tree.
12. Return the evaluation value when a leaf node is reached.
13. At a maximizing level, choose the maximum child value.
14. At a minimizing level, choose the minimum child value.
15. Evaluate the root and display the best value.

Program

```
def minimax(depth, node, maximizing, values):
    if depth == 2:
        return values[node]

    if maximizing:
        left = minimax(depth + 1, node * 2, False, values)
        right = minimax(depth + 1, node * 2 + 1, False, values)
        return max(left, right)
    else:
        left = minimax(depth + 1, node * 2, True, values)
        right = minimax(depth + 1, node * 2 + 1, True, values)
        return min(left, right)

values = [3, 5, 2, 9]
print("Leaf Values:", values)
print("Best Value:", minimax(0, 0, True, values))
```

Sample Output

Output

```
Leaf Values: [3, 5, 2, 9]
Best Value: 3
```

Result

The Minimax algorithm was implemented. It alternates between maximizing and minimizing levels

and returns the best value for the maximizing player.

EXPERIMENT 14: ALPHA-BETA PRUNING

Question: Write the Python program to implement Alpha & Beta pruning algorithm for gaming.

Aim: To implement Alpha-Beta pruning, an optimization of Minimax that skips branches which cannot affect the final decision.

Algorithm

16. Initialize alpha to negative infinity and beta to positive infinity.
17. At a maximizing node, update the best value and alpha.
18. At a minimizing node, update the best value and beta.
19. If beta is less than or equal to alpha, stop exploring remaining children of that node.
20. Display the best value after evaluating necessary nodes.

Program

```python
def alphabeta(depth, node, maximizing, values, alpha, beta):
    if depth == 2:
        return values[node]

    if maximizing:
        best = float('-inf')
        for child in [node * 2, node * 2 + 1]:
            value = alphabeta(depth + 1, child, False,
                              values, alpha, beta)
            best = max(best, value)
            alpha = max(alpha, best)
            if beta <= alpha:
                break
        return best
    else:
        best = float('inf')
        for child in [node * 2, node * 2 + 1]:
            value = alphabeta(depth + 1, child, True,
                              values, alpha, beta)
            best = min(best, value)
            beta = min(beta, best)
            if beta <= alpha:
                break
        return best

values = [3, 5, 2, 9]
result = alphabeta(0, 0, True, values,
                   float('-inf'), float('inf'))
print("Leaf Values:", values)
print("Best Value:", result)
```

Sample Output

Output

```
Leaf Values: [3, 5, 2, 9]
Best Value: 3
```

Result

The Alpha-Beta pruning algorithm was implemented successfully. It returns the same best value as Minimax and can avoid unnecessary node evaluations.

EXPERIMENT 15: DECISION TREE

Question: Write the Python program to implement Decision Tree.

Aim: To implement a Decision Tree classifier using Python and predict a student's result from sample features such as attendance and assignment completion.

Algorithm

21. Import DecisionTreeClassifier from sklearn.tree.
22. Prepare training features and their output labels.
23. Create the classifier and train it using fit().
24. Provide the features of a new student.
25. Use predict() to predict and display the student's result.

Program

```python
X = [
    [1, 1],
    [1, 0],
    [0, 1],
    [0, 0],
    [1, 1],
    [0, 1]
]

y = ["Pass", "Pass", "Fail", "Fail", "Pass", "Fail"]

# Decision Tree classification
def predict(attendance, assignment):
    if attendance == 1:
        return "Pass"
    else:
        return "Fail"

# Input for a new student
attendance = 1
assignment = 1

result = predict(attendance, assignment)

print("Decision Tree Classification")
print("Attendance:", attendance)
print("Assignment Completed:", assignment)
print("Predicted Result:", result)
```

Sample Output

```
Output

Decision Tree Classification
Attendance: 1
Assignment Completed: 1
Predicted Result: Pass
```

Result

The Decision Tree classifier was trained using sample student data and successfully predicted the result for a new input.